

U3 · Aula 3 — Testando SPAs Modernas

Hydration, Lazy Loading e React Server Components

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Hydration:** fases da SPA, waiting correto, RSC
- **Parte 2 — Padrões de teste:** lazy loading, rotas, error boundary, Suspense
- **Parte 3 — Prática guiada:** comportamento SPA do CineFav Web

Depois desta aula: screencast — eu resolvendo os specs passo a passo.

Objetivos de aprendizagem

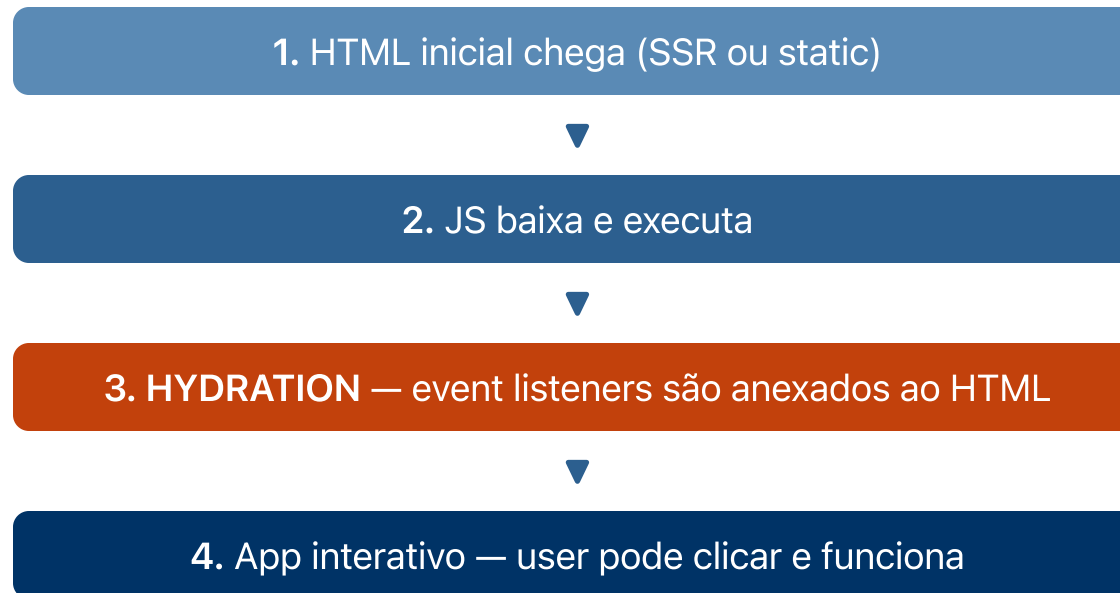
- **Explicar** por que clicar antes da hydration gera flake intermitente
- **Esperar** pelo sinal certo do app (marker), nunca por tempo
- **Testar** lazy loading, navegação client-side e deep links
- **Cobrir** error boundaries e loading states (Suspense)
- **Completar** o spec de comportamento SPA do CineFav Web

Parte 1 — Hydration

Fases da SPA · waiting correto · React Server Components

O problema das SPAs para testes

SPAs têm fases de execução que testes precisam respeitar:



Teste que clica em fase 3 antes de hydration completar → **flake intermitente em produção** que não aparece em dev.

Hydration testing — aguardar corretamente

```
test('botão funciona após hydration', async ({ page }) => {
  await page.goto('/');

  // networkidle: heurística de que tudo carregou + JS provavelmente hidratou
  await page.waitForLoadState('networkidle');

  // Para Next.js: aguardar marker explícito do framework
  await page.waitForFunction(() => (window as any).__NEXT_DATA__ !== undefined);

  await page.getByRole('button', { name: 'Click me' }).click();
  await expect(page.getByText('Clicado!')).toBeVisible();
});
```

`networkidle` é heurística — frameworks modernos expõem markers explícitos. Prefira o marker.

React Server Components — particularidades

RSC hidrata por boundary, não tudo de uma vez.

```
test('componente de usuário hidrata separado', async ({ page }) => {  
  await page.goto('/dashboard');  
  
  // Aguardar skeleton sumir (indica que RSC hidratou aquele boundary)  
  await expect(page.getByTestId('user-skeleton')).not.toBeVisible();  
  
  // Agora sim o componente está interativo  
  await page.getByRole('button', { name: 'Editar perfil' }).click();  
  await expect(page.getByRole('dialog')).toBeVisible();  
});
```

Parte 2 — Padrões de teste SPA

Lazy loading · rotas · error boundary · Suspense

Lazy loading — testar o que o usuário verá

```
test('imagem lazy carrega ao entrar no viewport', async ({ page }) => {  
  await page.goto('/galeria');  
  
  // Elemento fora do viewport — ainda não carregado  
  const image = page.getByTestId('lazy-image-5');  
  await expect(image).not.toBeInViewport();  
  
  // Scroll até o elemento  
  await image.scrollIntoViewIfNeeded();  
  
  // Aguardar carregamento  
  await expect(image).toHaveAttribute('src', /^https:/);  
  await expect(image).not.toHaveAttribute('loading', 'lazy'); // removido após load  
});
```

Route transitions — navegação SPA

```
test('navegação SPA não recarrega página', async ({ page }) => {  
  await page.goto('/');  
  
  // Capturar requests durante navegação  
  const requests: string[] = [];  
  page.on('request', (req) => requests.push(req.url()));  
  
  await page.getByRole('link', { name: 'Produtos' }).click();  
  await page.waitForURL('/produtos');  
  
  // SPA: navegação não deve recarregar HTML completo  
  const htmlRequests = requests.filter(  
    (url) => url.endsWith('/produtos') && !url.includes('json')  
  );  
  expect(htmlRequests).toHaveLength(0); // sem reload de página  
});
```

Error boundaries — testar recuperação

```
test('error boundary exhibe fallback e permite retry', async ({ page }) => {  
  // Fazer API falhar  
  await page.route('**/api/produto/*', (route) =>  
    route.fulfill({ status: 500 })  
  );  
  
  await page.goto('/produto/123');  
  
  // Error boundary deve capturar e mostrar fallback  
  await expect(page.getByText('Algo deu errado')).toBeVisible();  
  await expect(page.getByRole('button', { name: 'Tentar novamente' })).toBeVisible();  
  
  // Permitir a request e tentar novamente  
  await page.unroute('**/api/produto/*');  
  await page.getByRole('button', { name: 'Tentar novamente' }).click();  
  
  await expect(page.getByText('Produto')).toBeVisible();  
});
```

Suspense — testar loading states

```
test('suspense exhibe skeleton enquanto carrega', async ({ page }) => {  
  // Atrasar a response para garantir que Suspense apareça  
  await page.route('**/api/dados-pesados', async (route) => {  
    await new Promise(r => setTimeout(r, 500));  
    await route.continue();  
  });  
  
  await page.goto('/relatorio');  
  
  // Skeleton deve aparecer imediatamente  
  await expect(page.getByTestId('relatorio-skeleton')).toBeVisible();  
  
  // Conteúdo real deve aparecer após carregamento  
  await expect(page.getByTestId('relatorio-dados')).toBeVisible({  
    timeout: 5000,  
  });  
  await expect(page.getByTestId('relatorio-skeleton')).not.toBeVisible();  
});
```

Pitfalls comuns em testes de SPA

- ✗ Clicar sem aguardar hydration — falha intermitente que some no retry
- ✗ `waitForURL` como proxy de hydration — URL muda antes de hydration completar
- ✗ Testar só conteúdo renderizado no servidor — esconde bugs de interatividade
- ✗ Ignorar loading states (Suspense/skeleton) — UX crítica para usuários lentos
- ✗ `networkidle` como único indicador — apps com polling nunca ficam idle

Resumo

- **Hydration** é fase crítica — aguardar marker do framework, não só `networkidle`
- **RSC** hidrata por boundary — cada `<Suspense>` boundary tem seu próprio ciclo
- **Lazy loading** exige scroll explícito para disparar o load
- **Route transitions** SPA não recarregam HTML — validar isso nos testes
- **Error boundaries + Suspense** precisam de testes dedicados para UX em falha/loading

Parte 3 — Prática guiada

Comportamento SPA do CineFav Web

A prática — 04-spa.spec.ts

Mesmo app das aulas anteriores:

```
cd exercicios/02-lab-web-pwa-playwright/pratica
ls tests/e2e      # confirma: 04-spa.spec.ts está aí
```

O CineFav Web é uma SPA React Router com **rotas lazy** (`React.lazy`) e um **signal de pronto**: quando o React monta, o app seta `data-app-ready="true"` no `<html>` .

Prática 1 — esperar pelo sinal do app

```
// o app avisa quando está interativo:  
await page.locator('html[data-app-ready="true"]').waitFor();
```

- **Resolvida no screencast** — é o "marker do framework" da Parte 1
- Compare com a alternativa ruim: `waitForTimeout(3000)` — por que ela passa hoje e quebra amanhã?

Práticas 2–4 — os desafios

```
// 2. navegação client-side não recarrega:  
await page.evaluate(() => { (window as any).__spaMarker = 42; });  
// navega... e o marker sobrevive? full reload teria apagado.  
  
// 3. lazy chunk: colete os .js baixados com page.on('request')  
//     e prove que navegar até /favorites baixa um chunk NOVO.  
  
// 4. deep link: goto('/movie/603') direto → detail-title = "Matrix"
```

- Truque do `__spaMarker`: estado do JS como **testemunha** do reload
- Os TODOs guiam cada passo — termine os 4 testes

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Repo da prática** — link no Canvas, specs-guia prontos
- **Travou?** — `npm run test:e2e:ui` + aba Network do modo UI mostra os chunks

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith